

The Hidden Threat in Plain Text: Attacking RAG Data Loaders

Alberto Castagnaro*
University of Padua
Padova, Italy
alberto.castagnaro@unipd.it

Umberto Salviati*
University of Padua
Padova, Italy
umberto.salviati@studenti.unipd.it

Mauro Conti
University of Padua
Padova, Italy
Örebro University
Örebro, Sweden
mauro.conti@unipd.it

Luca Pajola
Spritz Matter
Padova, Italy
luca.pajola@spritzmatter.com

Simeone Pizzi
Spritz Matter
Padova, Italy
sime.pizzi@gmail.com

Abstract

Large Language Models (LLMs) have transformed human-machine interaction since ChatGPT's 2022 debut, with Retrieval-Augmented Generation (RAG) emerging as a key framework that enhances LLM outputs by integrating external knowledge. However, RAG's reliance on ingesting external documents introduces new vulnerabilities. This paper exposes a critical security gap at the data loading stage, where malicious actors can stealthily corrupt RAG pipelines by exploiting document ingestion.

We propose a taxonomy of 9 knowledge-based poisoning attacks and introduce two novel threat vectors—*Content Obfuscation* and *Content Injection*—targeting common formats (DOCX, HTML, PDF). Using an automated toolkit implementing 19 stealthy injection techniques, we test five popular data loaders, finding a 74.4% attack success rate across 357 scenarios. We further validate these threats on six end-to-end RAG systems—including white-box pipelines and black-box services like NotebookLM and OpenAI Assistants—demonstrating high success rates and critical vulnerabilities that bypass filters and silently compromise output integrity. Our results emphasize the urgent need to secure the document ingestion process in RAG systems against covert content manipulations.

CCS Concepts

• **Computing methodologies** → **Artificial intelligence**; **Natural language generation**; • **Information systems** → **Language models**; **Question answering**; • **Security and privacy** → **Data anonymization and sanitization**; **Penetration testing**.

Keywords

Retrieval Augmented Generation, RAG, Large Language Models, LLM, AI security, LLM security, Document Poisoning, Knowledge Base Poisoning

*Both authors contributed equally.



This work is licensed under a Creative Commons Attribution 4.0 International License.
AISEC '25, Taipei, Taiwan
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1895-3/25/10
<https://doi.org/10.1145/3733799.3762976>

ACM Reference Format:

Alberto Castagnaro, Umberto Salviati, Mauro Conti, Luca Pajola, and Simeone Pizzi. 2025. The Hidden Threat in Plain Text: Attacking RAG Data Loaders. In *Proceedings of the 2025 Workshop on Artificial Intelligence and Security (AISEC '25)*, October 13–17, 2025, Taipei, Taiwan. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3733799.3762976>

1 Introduction

Large Language Models (LLMs), despite their capabilities [4], suffer from limitations such as hallucinations, outdated knowledge, and the inability to access real-time or proprietary information [20], which Retrieval-Augmented Generation (RAG) addresses by integrating external knowledge sources [14]. By retrieving relevant documents from structured or unstructured data and integrating them into the generation, RAG improves the reliability and applicability of answers, making it particularly valuable for enterprise documentation, legal research, and cybersecurity operations.

While LLMs exhibit remarkable capabilities in various domains, they also pose significant and novel security challenges. Their reliance on massive datasets and complex architectures makes them vulnerable to a variety of attacks that can compromise their reliability and security. Prompt injection attacks can override system instructions and manipulate model behavior, resulting in unintended outcomes [19]. In addition, LLMs can generate biased or harmful content from training data [23] and are vulnerable to attacks like model inversion and adversarial examples that extract or manipulate sensitive information [5]. More recently, a Common Vulnerabilities and Exposures (CVE) has been disclosed in Meta's Llama framework exposed AI applications to remote execution attacks (CVE-2024-50050).¹

Contributions. Since RAG systems are becoming more and more popular, it is very important to understand how well they are protected against cyberattacks. For this reason, in this paper we propose a study of such systems from a cybersecurity perspective, with the three key contributions: (1) we propose a taxonomy of knowledge-based poisoning attacks against RAG; (2) we introduce a set of practical techniques for poisoning the knowledge base during data loading; (3) we evaluate both popular open-source data loading libraries and black-box RAG systems from popular providers, finding that most of them are vulnerable to simple techniques. In this

¹<https://nvd.nist.gov/vuln/detail/CVE-2024-50050>

regard, we develop and release PhantomText toolkit, a framework for the automatic generation of poisoned documents.

2 Background

Retrieval-Augmented Generation. Retrieval-Augmented Generation (RAG) is a framework that enhances large language models (LLMs) by integrating external knowledge retrieval, improving the relevance and accuracy of outputs [14]. Unlike traditional models, RAG accesses real-time information, making it effective in dynamic contexts.

A RAG pipeline integrates information retrieval with generative AI to improve response accuracy and relevance. It begins with *database generation*, where structured or unstructured data (e.g., documents, PDFs, web pages) are embedded and stored in a *vector database*². Upon receiving a *user query*, relevant documents are retrieved and the results are passed to a *language model*, which incorporates this context to produce accurate, informed responses. Building a vector database As our research focuses on the security of the *parsing stage* in RAG pipelines, it is crucial to examine how the vector database is built. The reliability of retrieval depends on how data is *preprocessed*, *tokenized*, *embedded*, and stored, as flaws in these steps may enable poisoning attacks, adversarial inputs, or inconsistent retrieval.

A RAG pipeline is constructed by (1) gathering relevant data; (2) loading the data, segmenting it into chunks; (3) converting the data into *vector embeddings*; (4) storing the data in a vector database for efficient retrieval; (5) retrieving the relevant chunks at query time and inserting them into the LLM context, to enhance its responses.

LLM Jailbreaking. LLM jailbreaking [21] refers to the act of bypassing or manipulating the safeguards built into a language model, which can lead to unauthorized data access and the generation of harmful content. RAG systems can be vectors for such attacks [8].

3 Taxonomy of Knowledge Base Poisoning Attacks in RAG Systems

With the growing adoption of AI systems, security researchers have started studying the security risks of RAG pipelines. To fully understand these risks, we highlight and categorize the goals of potential attackers. We define four main attack families: system integrity attacks, output manipulation attacks, knowledge poisoning attacks, and security and privacy attacks, based on the adversary’s objective. To ensure a systematic analysis, we map each attack to the CIA triad which has the advantage of being a familiar framework for most security researchers.

3.1 RAG and the CIA triad

RAG integrates information retrieval with generative models to improve content accuracy and relevance, yet its often-overlooked vulnerability to cyberattacks can be systematically analyzed using the CIA triad (Confidentiality, Integrity, and Availability).

Integrity concerns stem from the risk of retrieving or generating inaccurate, tampered with, or adversarial content. Compromised

or unreliable sources can lead to misinformation or biased outputs. Availability is equally critical, as RAG systems depend on external databases and APIs, making them vulnerable to denial-of-service attacks or retrieval failures. Securing RAG applications within the CIA framework requires robust access control, data validation, and resilience against adversarial input. Table 1 shows the mapping between the proposed attacks and the CIA triad.

Table 1: CIA Mapping for RAG Knowledge Base Poisoning Attacks

Attack	(C)	(I)	(A)
<i>System Integrity Attacks</i>			
Pipeline Failure			✓
Reasoning Overload			✓
<i>Response Manipulation Attacks</i>			
Unreadable Output			✓
Empty Statement Response			✓
Vague Output		✓	✓
<i>Knowledge Manipulation Attacks</i>			
Bias Injection		✓	
Factual Distortion		✓	
Outdated Knowledge		✓	
<i>Security and Privacy Attacks</i>			
Sensitive Data Disclosure	✓		

3.2 System Integrity Attacks

Attacks that disrupt RAG stability, performance, or efficiency.

3.2.1 Pipeline Failure (A). Injected adversarial data causes unexpected software failures, including crashes, infinite loops, or unhandled exceptions. It affects system availability, as the system may crash or enter an unstable state, making it inaccessible to users. We did not find references to RAG pipeline failures in the literature, however, Xiao et al. [24] have already discussed DoS attacks in traditional machine learning libraries.

🚨 **A1: Pipeline Failure.** Crashes or destabilizes the RAG system through adversarial inputs that trigger software failures, leading to a denial-of-service.

3.2.2 Reasoning Overload (A, I). A recent trend in Large Language Models is *reasoning models*, which consists in models that are explicitly trained to describe how they got to the result before producing a final answer to the user query, thus improving their performance across a number of tasks [11, 22].

An LLM Denial-of-Service (DoS) attack can leverage this “reasoning” by poisoning the vector database to inject malicious content that forces the LLM to spend excessive computational resources, thereby slowing down inference and making the system less responsive, or even unresponsive, under certain conditions. The attack could impact multiple aspects:

- **Performance Slowdown (A):** The attack can result in a significant increase in processing time, severely degrading the user

²A vector database stores high-dimensional embeddings to enable fast semantic similarity searches.

experience. It impacts the Availability as the system remains functional but becomes inefficient, reducing usability.

- **Excessive Token Production (A, I):** The inclusion of decoy reasoning problems increases the token count, leading to a higher computational load. This amplifies the cost of operating the system, as each inference requires more tokens to be processed, which can negatively impact the financial sustainability of third-party applications that rely on reasoning LLMs. It affects availability, as an increased computational burden can slow down the system or cause request failures. It affects the integrity, as the attack manipulates the system's expected behavior by forcing unnecessary reasoning steps, altering the intended flow of information processing.
- **Resource Strain and Service Denial (A):** Over time, if multiple users are subjected to similar poisoned queries, the system could become unresponsive, or the service could fail to return a timely response, essentially causing a DoS effect.

This attack aligns with the “OVERTHINK” attack [12], where adversaries inject decoy reasoning problems to increase token processing during inference, causing the model to “overthink” while generating responses.

🔗**A2: Reasoning Overload.** Injecting adversarial content designed to expand excessively upon retrieval, causing increased inference time and resource exhaustion.

3.3 Output Manipulation Attacks

Attacks that reduce response quality, clarity, or usability.

3.3.1 Unreadable Output (A). The system may produce nonsensical, illegible, or unusable outputs due to tokenization errors, corrupted embeddings, or external tampering. In some cases, adversarial perturbations can subtly modify inputs to exploit these vulnerabilities. For instance, attackers may inject misleading Unicode characters or embed Base64-encoded segments, formats that are technically valid but not interpretable by the model, causing garbled text, random character sequences, misinterpreted tokens, or otherwise irrelevant output. These manipulations degrade output interpretability without crashing the system, leaving it operational yet functionally unusable.

🔗**A3: Unreadable Output.** Injecting adversarial text containing excessive special characters or encoding artifacts, leading to responses filled with unreadable symbols.

3.3.2 Empty Statement Response (A). The attack targets output generation, causing the system to produce grammatically correct but meaningless responses that lack actionable context, often due to prompt filters, training-data biases, or safety constraints. For example, adversarial input might manipulate a RAG system to output generic fallback statements like “I’m sorry, I cannot provide that information,” even when the query is legitimate and should have resulted in a specific response. Attackers can exploit this vulnerability by feeding misleading or irrelevant data into the vector database and forcing the system into evasive or empty responses,

undermining its utility in critical contexts. This attack affects availability, as the system responds but provides no meaningful value, disrupting usability.

🔗**A4: Empty Statement Response.** Causing the system to respond with generic statements such as “I cannot answer this question.”

3.3.3 Vague Output (A, I). The system generates vague or misleading responses that lack clear factual grounding. These responses often lack the necessary specificity, context, or detail needed to effectively address the user’s query. For example, a RAG-powered assistant might respond with “There are many factors to consider,” when asked for a recommendation or solution, leaving the user unsure of what factors are involved or how they should proceed. Such responses often result from adversarial interference that forces the model to retrieve or generate generalized, context-agnostic text, which could be interpreted in numerous ways. This ambiguity undermines the system’s reliability and hinders users from making informed decisions, especially in environments where precision is key. This attack effect is twofold: (i) availability, as the model responds but does not deliver actionable insights, making the response functionally useless; (ii) integrity, as the system’s correctness is affected, as it no longer provides clear and accurate information.

🔗**A5: Vague Output.** Poisoning the knowledge base with subtly inconsistent data results in outputs like “There is no definitive answer to this question.”

3.4 Knowledge Manipulation Attacks

Attacks altering knowledge, affecting accuracy, bias or timeliness.

3.4.1 Bias Injection (I). The knowledge base is poisoned with biased content, which reinforces existing perspectives while reducing access to diverse viewpoints. This causes the RAG system to present skewed, unfair, or ideologically extreme narratives, affecting system integrity, as the responses become unreliable due to systemic bias in the retrieved information.

🔗**A6: Bias Injection.** Poisoning the RAG system with misleading information leading to biased responses favoring one perspective.

3.4.2 Factual Distortion (I). The attack occurs when the system generates or retrieves information that is factually incorrect, misleading, or unverifiable, thus compromising the system’s credibility and reliability. The attacks affect system integrity, as factual correctness is no longer guaranteed, corrupting the system’s reliability. Factual distortion can be linked to the more general phenomenon of misinformation campaigns (fake news) [13].

🔗**A7: Factual Distortion.** Modifying the knowledge base with factually incorrect information, leading the RAG system to propagate misinformation.

3.4.3 Outdated Knowledge (I). This attack obfuscates recent data, causing the RAG system to retrieve outdated information like obsolete studies, rulings, or product specs. Since the system still responds, outdated info may go unnoticed, leading to faulty recommendations or decisions.

A8: Outdated Knowledge. Causing the model to base its answers on obsolete knowledge, leading to responses containing inaccurate or outdated information.

3.5 Security and Privacy Violations

Attacks that breach confidentiality or expose sensitive data.

3.5.1 Sensitive Data Disclosure (C). Sensitive data disclosure occurs when the system inadvertently reveals confidential information breaching privacy and confidentiality. This can happen when the system retrieves or generates real data that should not be accessible or exposed to users. The disclosed data could include personally identifiable information (PII), such as phone numbers, medical records, financial information, or even sensitive corporate data. The attack impacts system confidentiality by improperly disclosing sensitive data, causing privacy and security violations.

A9: Sensitive Data Disclosure. Making the RAG system disclose sensitive information, such as PII or corporate data, to unauthorized users.

4 Threat Model

Target. The *Target* of the attack is a RAG system (Section 2) operated by a business. We assume that before documents are added to the knowledge base of the RAG, they are inspected by one or more employees, using tools such as PDF readers and web browsers to check for abuses. We assume that these employees are not IT experts, but they are experts in the topics covered by the documents, i.e., they are able to recognize documents that contain inaccurate or false information. We call these employees *Inspectors*.

Attacker. The *Attacker* is a malicious actor that does not have direct access to the *Target*, but is able to inject poisoned documents into the RAG knowledge base by different means. These include, but are not limited to:

- **Supply Chain Attacks:** The attacker could be a service provider, and poison all the documents it provides as documentation to its clients. In this case, any of the clients that imports the documents in a RAG system would become a victim.
- **Web-Based Poisoning:** Attackers manipulate public sources (e.g., wikis, research databases) by injecting false or biased information. A RAG-powered cybersecurity assistant, for example, could retrieve and suggest insecure cryptographic practices, exposing developers to security risks.
- **Insider Threats in Organization:** A malicious employee in a company modifies internal documents used by a RAG system, injecting false compliance guidelines. Employees

relying on the system may unknowingly follow misleading legal or financial advice, leading to regulatory violations.

Goal. The attacker seeks to undermine the integrity, reliability, or security of RAG systems by embedding imperceptible manipulations in retrieved knowledge. Their objectives include misleading responses that introduce factual distortions or biases, stealthy information manipulation via obfuscated content, evading detection mechanisms through adversarial encoding, targeted prompt manipulation to exploit hidden triggers, and poisoning the knowledge base to degrade system performance. A detailed breakdown of these goals and their mapping to CIA principles is provided in Section 3.

To achieve these objectives, the attacker leverages vulnerabilities in the *Data Loading* stage of the RAG pipeline, performing attacks that fall under two broad categories:

- **Content Obfuscation:** Disrupts or distorts existing information in the document using invisible characters, making it harder for models to correctly extract or interpret the original content.
- **Content Injection:** Inserts new, invisible concepts into the document, effectively introducing misleading or fabricated knowledge into the knowledge base.

5 Data Loading Deception Techniques

In this section, we describe the techniques we studied to attack the data loading stage of a RAG pipeline.

5.0.1 Technique Organization. The attack strategy is organized into two major deception technique families:

- **Content Obfuscation:** This family of attacks aims to disrupt or distort existing information in the document, reducing its readability or interpretability for the AI model. We use techniques such as zero-width characters (e.g., ZWSP, ZWNJ), homoglyph substitutions (e.g., replacing letters with visually similar Unicode characters), and bidirectional (Bidi) reordering to subtly alter the structure of words or sentences without changing their appearance to the human eye.
- **Content Injection:** These attacks introduce new, invisible or hidden concepts into the document, effectively poisoning the knowledge base with misleading or fabricated facts. Techniques include using font size zero to make text visually disappear, positioning text out of the visible margin, and injecting data into document metadata fields (e.g., PDF/XMP metadata), which may be parsed and indexed by AI systems but remain invisible to end users.

In our analysis – as RAG can be set up to handle input documents of different formats – we examine the structure and formatting capabilities of widely used document types— PDF (📄), HTML (🌐), and DOCX (📑)—to identify potential threat vectors that enable both content obfuscation and content injection attacks. The rich styling and metadata features of these formats create ideal surfaces for content obfuscation or injection - using invisible characters, style tweaks, embedded objects, and hidden fields - and analyzing their text rendering, metadata handling, and layout exposes unique stealth attack vectors in RAG pipelines.

5.1 Content Obfuscation

5.1.1 Diacritical Marks.   Diacritical marks are small signs or symbols added to letters to alter their pronunciation, meaning, or distinguish between words in different languages. They include accent marks (e.g., é, ñ), tildes, umlauts, and cedillas, and are commonly used in languages like French, Spanish, and German. This attack involves combining diacritical marks—Unicode characters that modify a base character—to create visually complex representations of text. In our attacks, we add ten diacritical character to a random letter on the target word.

5.1.2 Homoglyph Characters.   This attack replaces characters with visually identical but semantically different Unicode homoglyphs, such as substituting Latin “A” (U+0041) with Cyrillic one (U+0410). Homoglyph substitution attacks have been shown to be effective in attacking traditional ML-based and information-retrieval systems deployed in real-world applications [2, 3, 9].

5.1.3 OCR-Poisoning.   This attack involves introducing subtle perturbations into text parsed by Optical Character Recognition (OCR) systems, particularly in images and complex PDFs. By manipulating visual features that OCR relies on for text extraction, these perturbations can confuse the recognition process, leading to incorrect or misleading interpretations of the original text. This attack has been demonstrated successful in modern OCR systems [7].

5.1.4 Reordering Characters.   A reordering attack, or bidirectional (Bidi) attack, leverages the Unicode Bidirectional algorithm to reorder characters visually while preserving a different logical encoding order. By strategically embedding right-to-left (RTL) characters within left-to-right (LTR) text, it creates discrepancies between human and machine interpretation. Bidi attacks have been demonstrated successful in attacking traditional ML-based and information-retrieval systems deployed in real applications [2, 3].

5.1.5 Zero-Width Characters.   This attack leverages zero-width (ZeW) characters—Unicode symbols that take no visible space but alter text representation. Common examples include Zero-Width Space (U+200B), Zero-Width Non-Joiner (U+200C), and Zero-Width Joiner (U+200D). These characters are typically used for text formatting, preventing ligatures, or controlling word breaks in multilingual settings. By inserting them strategically, the attack disrupts tokenization and downstream NLP processing without altering human-readable text. Zero-width space attacks have been demonstrated successful in attacking traditional machine learning-based and information-retrieval systems deployed in real applications [2, 3, 18]. We implemented three distinct variants, conducting tests over the 19 distinct ZeW characters:

- (1) *Mask1*  : Insertion of one ZeW character in the middle of the target word.
- (2) *Mask2*  : Insertion of three ZeW characters in the middle of the target word.
- (3) *Mask3*  : Insertion of many ZeW characters between every character of a target word.

5.2 Content Injection

5.2.1 Camouflage Element.

The technique embeds text behind other elements within a document, effectively rendering it invisible to the user while preserving

its presence within the document’s structure. In this work, we inject malicious text behind images.

5.2.2 Metadata.   This attack involves adding or altering a document’s metadata with malicious or poisoned data that may be inadvertently parsed and processed by document loaders or AI models. We inject random or adversarial elements into the document’s metadata (i.e. PDF metadata or HTML <head>).

5.2.3 Out-of-Bound Text.

This attack involves injecting text outside the visible area of the document, such as beyond the margins or in hidden portions of the webpage or document. While the text becomes invisible to the user because it is rendered outside the viewport, it is still processed by machines. We designed two variants:

- (1) *Random position*  : The text is placed at random coordinates outside the visible area of the document or webpage, such as the top-left or bottom-right corners.
- (2) *Right margin* : The text is aligned to the far right, partially pushed off-screen, leaving only a fragment visible.

5.2.4 Transparent Text.   This attack makes adversarial text the same color as the background, making it invisible to humans while still being processed by AI models. It exploits formatting features in different file types (e.g., HTML, DOCX, PDF).

- (1) *Font color matching the background*  , with background set to white in the Word format case.
- (2) *Opacity 0*  : In HTML, setting ‘opacity: 0;’ in CSS (‘hidden text’) makes text invisible to users while still readable by screen readers and AI models. In PDFs, this effect is replicated using transparent text layers or annotations with zero opacity.
- (3) *Opacity 0.01*  .
- (4) *Visibility* : In HTML, using ‘visibility: hidden;’ (‘hidden text’) makes the text invisible, but unlike ‘opacity: 0;’, it also prevents the element from taking up space in the layout. However, the text remains in the HTML source and can still be processed by AI models, crawlers, or screen readers, making it another potential technique for adversarial attacks.
- (5) *Vanish* : In DOCX, the “Vanish” property hides text from being displayed in the document while keeping it in the file structure, allowing it to be processed by text parsers and AI models. This can be applied using Word’s formatting options (‘<w: vanish/>’ in XML) or by selecting “Hidden” in the font settings, making it a potential method for adversarial attacks.

5.2.5 Zero-Size Font.   This method reduces the font size of adversarial text to zero or near-zero, making it imperceptible to users but still interpretable by machine learning models. It exploits text rendering differences to introduce hidden prompts or evade detection in AI-driven systems. We implement the following variants of the attack:

- (1) *Size0*  : font size set to 0, making it invisible.
- (2) *Size0.01*  : font size set to 0.01, making it nearly invisible.
- (3) *Size1* : font size set to 1, making it very small. This is the minimum size accepted by Word documents.

5.3 Two-fold Effect

We now present techniques that can be used both for content obfuscation and content injection.

5.3.1 Font Poisoning.  This attack utilizes a custom font to map displayed characters to different underlying characters parsed by document loaders. For example, in a specially designed font, the glyph for the character “A” could be associated with the ASCII code for “B” (U+0042). This manipulation creates a visual representation that appears as “A” to human readers, while the underlying text is actually parsed as “B” by automated systems, compromising the integrity of text processing. The attack effect can be two-fold:

- (1) Semantic disruption, as the attacker might substitute the ASCII representation of glyphs to random characters. For instance, the word “apple” could be represented by the ASCII representation “qwerty”,
- (2) Invisible text injection, as the attacker might substitute the ASCII representation of glyphs to target characters. For instance, the word “apple” could be represented by the ASCII representation “drugs” (note that multiple fonts can be used to map the same glyph to multiple ASCII representations).

Font poisoning has been introduced in [16], where authors demonstrated the feasibility of PDF content masking against real-world systems, including Turnitin, Bing, Yahoo!, and DuckDuckGo.

6 Evaluation

As part of our contribution, we introduce the **PhantomText Toolkit**, a modular framework for crafting invisible manipulations in documents (DOCX, HTML, PDF). PhantomText supports the attack strategies introduced in Section 5. The toolkit is open-source and available at the paper repository: <https://github.com/pajola/PhantomText-Paper>.

This repository contains all material necessary to reproduce our experiments. To evaluate the effectiveness of PhantomText, we designed three experiments targeting the end-to-end pipeline of RAG systems.

- **Experiment 1: Poisoning Dataloaders.** We apply all PhantomText techniques to HTML, DOCX, and PDF documents using popular document loader implementations, assessing whether invisible artifacts can survive ingestion and be embedded into the knowledge base. This demonstrates that PhantomText techniques can poison the knowledge base by manipulating the content of documents at the data loader stage.
- **Experiment 2: End-to-End RAG Manipulation.** Poisoning the knowledge base does not necessarily guarantee that the RAG system will be impacted in the same way, since the LLM model that generates the output may be resilient to such attacks. For this reason, we study the full end-to-end RAG pipeline, to determine if the invisible manipulations in the poisoned documents influence the retriever and the generative model.
- **Experiment 3: CIA triad-oriented RAG attacks.** Finally, we demonstrate how the techniques we described can be used by malicious actors to perform the attacks described in Section 3. We composed targeted attacks using the most effective techniques from the toolkit, showing that PhantomText enables subtle, high-impact attacks.

In the following sections, we first present the research question that motivates our experiments, then detail the methodology we followed to perform the experiments, and finally report the results we obtained, with an answer for the research question.

6.1 Exp1: Poisoning Data Loaders

6.1.1 Overview. Here, we assess the efficacy of PhantomText in poisoning the RAG systems’ knowledge base by targeting the document parsing frameworks’ Data Loader components. It tests if invisible manipulations, such as obfuscation and injection, can compromise ingestion of HTML, DOCX, and PDF documents; results show PhantomText effectively embeds malicious artifacts to poison RAG knowledge bases.

 **RQ:** Are popular Document Loaders vulnerable to PhantomText toolkit content obfuscation and injection techniques?

6.1.2 Experimental Settings.

Document Loaders. We tested the following frameworks: Docling [1, 15], Haystack, LangChain, LlamaIndex, and LLMSharpa.³ These frameworks were selected based on their popularity, trending status on GitHub, and widespread adoption in RAG development. For each framework, we focused on the document parsers responsible for handling different file formats, i.e. PDF, HTML, and DOCX. In total, 21 different parsers across the selected frameworks were evaluated. Specifically, for Docling we used the Default parser, the PyPDFium parser, the HTML parser, the DOCX parser, and the OCR parser; for Haystack, the PyPDF parser, the PDFMiner parser, the HTML parser, and the DOCX parser were evaluated; for LlamaIndex, we assessed the LlamaParse, SimpleDirectoryReader, and HTML-TagReader; for LangChain, the PyPDF (with and without OCR), PDFPlumber, PyPDFium2, PyMUPDF (with and without OCR), UnstructuredHTML, BShtml, and Docx2txt parsers were employed; finally, the LLMSharpa Default parser was evaluated.⁴

Dataset. We created a custom dataset to test PhantomText’s ability to obfuscate and inject content into DOCX, HTML, and PDF files. The dataset is created by utilizing 100 random samples from Amazon Reviews’23 [10]. Then, starting from each sample, we generate a poisoned document as follows: for *content obfuscation* techniques, a random word is selected within the document text and replaced with the injection counterpart; in the *content injection*, we insert a random word not present in the document.⁵ This procedure is repeated for all the various techniques we presented in Section 5, generating a dataset of 4200 documents. In other words, our dataset now contains those 100 original samples repeated with different injection techniques (e.g., 100 documents are obfuscated with *zero-width character mask1*, 100 with *zero-width character mask2*, and so on). We test each document loader configuration with such a dataset, resulting in a total of 35,900 evaluations.

It is important to clarify that our evaluation focuses on the robustness of document ingestion pipelines—software components

³We provide the list of software in the Appendix A.2

⁴The OCR we used was tesseract for all the loaders.

⁵The random word is picked from this list: <https://www.kaggle.com/datasets/ratatman/english-word-frequency>.

responsible for parsing and encoding input documents prior to language model inference—rather than on the performance or generalization of ML models themselves. These preprocessing algorithms are domain-agnostic, operating at a syntactic and structural level regardless of semantic content or text domain. Thus, Amazon Reviews provide a controlled yet realistic dataset, suitable for systematically evaluating injection attacks on software vulnerabilities.

Evaluation. A single evaluation is considered successful if, in the *Content Obfuscation* scenario, the target word was completely absent from the parsed text, and in the *Content Injection* scenario, the synthetic word was correctly present in the parsed output. The **Attack Success Rate (ASR)** is defined as the ratio between the number of successful tests and the total number of attempts, i.e.,

$$ASR = \frac{\text{Success}}{\text{Success} + \text{Failed}}$$

6.1.3 Results. We summarize the key findings in this section. The complete results, detailing the attack success rate for each configuration, can be found in the paper public repository.

Overall Trends. PhantomText Toolkit demonstrated an overall success rate of 74.4%, indicating a significant degree of effectiveness in compromising the integrity of RAG’s document loaders. Furthermore, out of 375 tests conducted, 238 yield a success rate over 95%. When analyzed by attack family, the performance shows notable distinctions between the two primary strategies. The content obfuscation and injection families achieved, respectively, a success rate of 76.7% and 72.4%.

Document Loaders Level. Figure 1a visualizes the varying effectiveness of the attacks across different data loaders. With an ASR above 0.6 for all loaders, the attack proves effective in manipulating document processing pipelines. Langchain shows the highest ASR, indicating significant vulnerability, while LlamaIndex exhibits the lowest ASR, demonstrating more resistance. Docling, LLMSherpa, and Haystack fall in the mid-range, showing moderate susceptibility. Overall, the results highlight that the attack is effective across all tested loaders, underscoring the need for improved defenses against such adversarial manipulations.

File Format. Figure 1b illustrates the effectiveness of the Text Injection attack across different file formats. All formats exhibit ASRs above 0.6, indicating that the attack is effective in each case. DOCX shows the highest ASR at 0.89, suggesting a high level of vulnerability. Overall, the results highlight that while the attack is effective across all file formats, DOCX is the most vulnerable, and PDF offers the best defense among the tested formats.

Success Rate of each Attack Technique. We conclude by analyzing the impact of each poisoning technique on document loaders, as shown in Figure 2. The results reveal that not all techniques are equally effective. For example, *font poisoning* and *homoglyph characters* consistently disrupt all data loaders tested, achieving a 100% attack success rate ($ASR = 1.0$). In contrast, *diacritical injection* shows limited effectiveness, with an average ASR of approximately 0.4. *Metadata injection* proves to be largely ineffective across the board, except for Langchain, where it reaches a moderate ASR of 0.5. The analysis also highlights significant variability in how

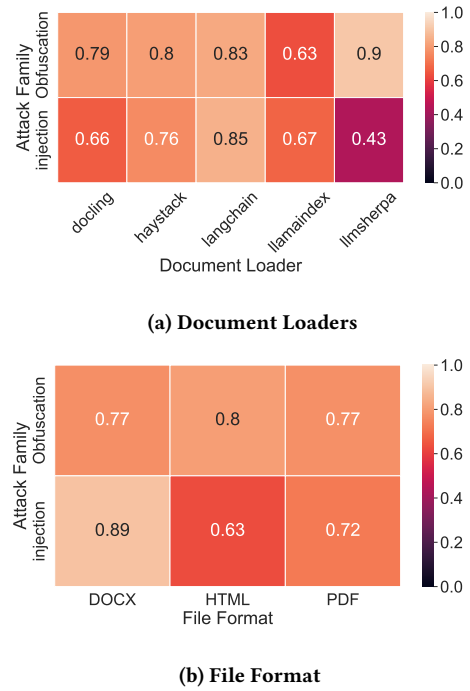


Figure 1: Attack Success Rate at varying poisoning families – from both *Semantic Corruption via Obfuscation* (in blue) and *Invisible Content Injection* (in orange) families – across different data loaders (left) and file format (right).

different data loaders respond to specific attacks. For example, *out-of-bound text* is particularly effective against Docling ($ASR = 0.83$) but less so against LlamaIndex ($ASR = 0.5$). In contrast, LlamaIndex is more vulnerable to *zero-size font* ($ASR = 0.81$), while Docling demonstrates greater resistance to the same technique ($ASR = 0.39$). These findings indicate that attack effectiveness is highly technique-dependent and that document loaders exhibit varying levels of resilience. Therefore, the choice of technique must be tailored to the specific target in order to maximize adversarial impact.

6.2 Exp2: End-to-End RAG Manipulation

6.2.1 Overview. Building on the results from Experiment 1, we explore whether a poisoned knowledge base affects the behavior of RAG systems. Specifically, we focus on determining whether the invisible manipulations introduced into the documents during ingestion can propagate through the retriever and generative model components of popular RAG frameworks. While the knowledge base is successfully poisoned in Experiment 1, the impact on system behavior is more nuanced, with some systems showing greater resilience than others. For simplicity, we focus only on PDF documents; however, the results can be generalized to other document formats, such as DOCX and HTML. This experiment demonstrates that document poisoning does not always guarantee that the entire RAG pipeline will be affected, and shows which techniques are the most effective in attacking different RAG pipelines.

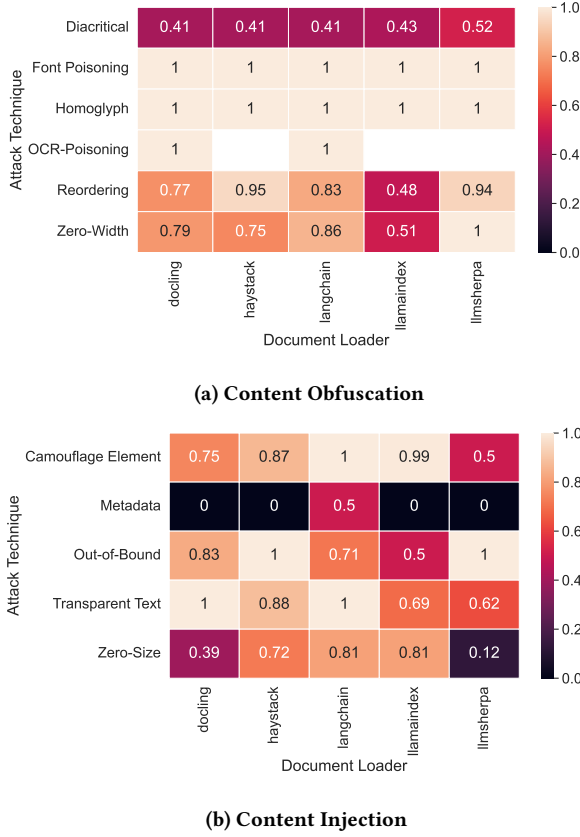


Figure 2: Attack Success Rate at varying poisoning families (Content Obfuscation and Injection).

Yes, Popular Document Loaders are severely vulnerable.

We observed that both injection and obfuscation techniques persist through preprocessing and are embedded into the vector store, enabling successful poisoning across multiple formats.

RQ: Can PhantomText toolkit content obfuscation and injection techniques affect end-to-end RAG pipelines? Which techniques are the most effective?

6.2.2 Experimental Settings.

RAG Configuration. In this work we tested different end-to-end RAG frameworks, with a total of 6 different setups ($M = 6$):

- **White-box** setting. Open-source frameworks that give us the capability to fully access the internal components, including the retriever and the language model. We utilized three distinct LLMs: Llama 3.2 (3B), Gemma 3 (27B), and Deep Seek R1. The RAG utilizes a local retriever. We locally implement 3 distinct RAG utilizing the official LangChain documentation

and a Chroma retriever⁶. A detailed description of our setup is reported in Appendix A.3.

- **Black-box** setting. It involves using pre-built RAG systems via APIs or web interfaces. We tested two black-box RAGs: OpenAI assistants (gpt-4o and o3-mini) and NotebookLM, based on Google’s Gemini 2.0 Flash.

Two LLMs – Deep Seek R1 and OpenAI o3-mini – were trained for reasoning responses.

In order to maximize fairness and ensure the validity of our RAG pipeline, for 5 out of 6 RAGs, we integrate a custom *system prompt* sourced directly from the OpenAI Playground interface, as reported in Appendix A.3. This prompt is notably more rigorous than Langchain’s default version; we adopted it to better simulate real-world scenarios and minimize bias in our experiments. The only exception is NotebookLM, which operates as a fully black-box system and does not allow users to customize the system prompt.

Dataset. The aim of this experiment is to understand if it is possible to affect end-to-end RAGs with PhantomText techniques, and to do so we deploy a straightforward test. We start from a benign document containing information about a fictional company, ensuring that RAG responses are not conditioned by the prior knowledge of the LLMs. Then, we produce 14 poisoned documents ($D = 14$) - one for each PDF PhantomText technique - in two fashions: for content obfuscation families, we use the techniques to hide specific information, while for the content injection, we add a new concept related a fictional competitor company.

Evaluation. We first evaluate the base quality of the RAG systems to ensure the validity of the experiment. We utilize a benign document and we generate queries to ask about the fictional company and its competitor ($Q = 4$). We test the 6 RAG systems against these queries, and verify that they are able to correctly answer. We repeat each query 10 times, to reduce randomness ($R = 10$). In total, we conduct $Q \times R \times M = 240$ tests. Then, we analyze the effectiveness of PhantomText toolkit, and its ability to obfuscate and inject information, by repeating the queries with poisoned documents. In this case, our total tests are $D \times R \times M = 840$ tests. All responses are manually evaluated to check whether the attack was successful.

6.2.3 Results. The quality assurance test resulted in 100% of accuracy, meaning that the RAG systems were able to correctly answer the queries based on the contents of the provided documents. As for the attack techniques, we report our results in Figure 3. We note that distinct patterns emerge across the various techniques. Most attacks implemented in PhantomText – such as *camouflage element* and *font poisoning* – consistently lead to effective RAG manipulation ($ASR = 1.0$) across all tested RAG systems, in both white-box and black-box settings. However, a small number of techniques such as *metadata* show no effect ($ASR = 0$). Finally, *reordering* and *homoglyphs* generate a range of outcomes, with some models achieving close to full success, while others, exhibiting reduced or null susceptibility. These findings highlight the varying degrees of vulnerability of different AI models to specific attacks. Due to space limitations, a detailed table breaking down the attack evaluation is provided only in the public repository.

⁶<https://python.langchain.com/docs/tutorials/rag/>

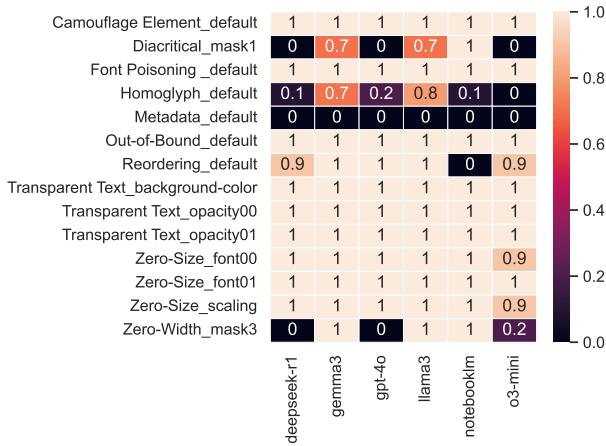


Figure 3: Experiment 2 – End-to-End RAG Manipulation.

Yes, RAG systems can be affected by PhantomText techniques. Not all PhantomText techniques are equally effective for RAG end-to-end systems; most are always effective (e.g., font poisoning), others consistently fail (e.g., metadata), and some are effective only on specific RAG systems (e.g., homoglyphs).

6.3 Exp3: CIA triad-oriented RAG attacks

6.3.1 Overview. We finally apply the most effective PhantomText techniques to craft targeted attacks that exploit vulnerabilities in RAG systems presented in Section 3, with a focus on the *Confidentiality*, *Integrity*, and *Availability* (CIA) triad. Using popular RAG frameworks, we demonstrate how invisible manipulations can lead to attacks such as leaking sensitive information, altering retrieved facts, and degrading system functionality. While we focus on PDF documents for simplicity, these techniques can be transferred to other document formats like DOCX and HTML. These attacks show that the combination of invisible content and subtle manipulations can be leveraged to bypass typical detection mechanisms, posing significant risks to both the integrity and availability of the system.

RQ: Can PhantomText toolkit techniques be leveraged to perform the attacks described in Section 3, affecting the confidentiality, integrity, and availability of state-of-the-art RAG systems?

6.3.2 Experimental Settings.

Datasets. We conduct attacks on the same RAG systems presented in Section 6.2. For each of the nine attack scenarios listed in Section 3, we crafted a series of datasets with different scenarios. For instance, when performing *sensitive data disclosure*, we designed a dataset containing sensitive information. As a representative case from the public repository (Appendix A.4), we detail one example to illustrate the attack’s mechanics and impact. In general, we defined 4 distinct dataset per scenario, with a varying number of documents ($N = \{1, 100\}$) and type of content (i.e., poisoned and unpoisoned).

Unpoisoned datasets are utilized as a control group to verify the correct execution of RAGs (i.e., if the RAG is able to answer the query without any tampering), while poisoned datasets are utilized to assess PhantomText attacks. In particular, for poisoned dataset, we are interested in two cases:

- **Single Poisoned Document.** The knowledge base contains only a single poisoned document. This scenario is designed to assess the maximum effectiveness of a specific attack when the adversary has complete control over the information source available to the RAG system.
- **100 Poisoned Documents.** The knowledge base contains all 100 documents (i.e. 99 legitimate and 1 poisoned). This scenario aimed to evaluate the attack performance in a more realistic setting where the attacker’s ability to manipulate the knowledge base is limited.

For each dataset and attack scenario, we designed ad-hoc queries.

Evaluation. We evaluate each attack scenario across the 6 RAG systems, using an LLM-as-a-judge to label the answers given by the RAG systems as either successful attacks or failures. The only exceptions are attacks a3, which was manually analyzed, and a9, where we used a simple substring match to look for leaked information in the responses. More details about our judging approach for the rest of attacks are reported in Appendix A.1. Each experiment is repeated 10 times to reduce randomness due to the non-deterministic nature of LLMs (with the exception of *pipeline failure* where we execute the test one time since we need to understand if the end-to-end RAG “crashes” and does not involve LLMs). For each attack, we report the number of successful executions out of 10, except for *pipeline failure* (single test) and *reasoning overload*, where we report the average increase factor in tokens used during reasoning.

6.3.3 Results. The results of the attacks are presented in Table 2, and highlight distinct vulnerabilities of RAG systems when targeted by PhantomText attacks in both white-box and black-box settings. A consistent pattern of high attack success rate is observed in several failure modes, especially in white-box scenarios. This indicates a critical lack of robustness in these systems against semantic disruption techniques that can affect confidentiality, integrity, and availability. Interestingly, even advanced models like deepseek-r1 are very susceptible to all executed attacks.

Black-box models display a more heterogeneous vulnerability profile. For instance, **NotebookLM** demonstrated strong resilience to several attack vectors, with a 0% success rate in attacks like *factual distortion*, *empty statement response*. On the other hand, commercial models such as **gpt-4o** and **o3-mini** were significantly more vulnerable, with high success rates (up to 10/10) in many categories. Specifically, attacks targeting **o3-mini** were always successful for 9 out of 16 scenarios. The results suggest that while some black-box models may benefit from stronger internal filtering or post-processing mechanisms for some attacks, they remain in general as vulnerable as their white-box counterparts. Importantly, the *reasoning-overload* attack, which measures token inflation, showed a very noticeable increase in response length for certain models (e.g., 4.9× in **o3-mini**), indicating that resource abuse or inefficiency can also be induced adversarially.

	deepseek-r1		White-box gemma3		llama		NotebookLM		Black-box gpt-4o		o3-mini	
	1	100	1	100	1	100	1	100	1	100	1	100
a1-pipeline-failure	1/1	1/1	1/1	1/1	1/1	1/1	0/1	0/1	0/1	0/1	0/1	0/1
a2-reasoning-overload	1.25	2.04	/	/	/	/	/	/	/	/	4.90	3.48
a3-unreadable-output	5/10	4/10	0/10	0/10	0/10	0/10	0/10	8/10	9/10	1/10	9/10	5/10
a4-empty-statement-response	10/10	10/10	10/10	10/10	10/10	10/10	0/10	0/10	8/10	8/10	10/10	8/10
a5-ambiguous-output	10/10	10/10	10/10	10/10	10/10	10/10	10/10	10/10	10/10	10/10	10/10	10/10
a6-bias-injection	10/10	10/10	10/10	10/10	9/10	10/10	0/10	9/10	7/10	9/10	10/10	10/10
a7-factual-distortion	10/10	10/10	10/10	10/10	10/10	10/10	0/10	0/10	10/10	10/10	10/10	10/10
a8-outdated-knowledge	10/10	10/10	10/10	10/10	10/10	10/10	10/10	10/10	9/10	8/10	10/10	10/10
a9-sensitive-data-disclosure	10/10	10/10	10/10	10/10	0/10	0/10	/	/	0/10	0/10	8/10	9/10

Table 2: Experiment 3 – Real-World Exploitation Across RAG Attack Scenarios. PhantomText attack success in executing different attacks to different RAG end-to-end systems, with 1 and 100 documents. Note that we report the attack success rate for each row, while for the attack “reasoning overload” we report the average factor of increase in the number of tokens.

A separate discussion can be made about the *pipeline failure* attack, since the results show that only our white-box RAG pipelines were vulnerable to this attack. First of all, we need to point out that we only tested a very simple attack vector, i.e. a very big PDF document, which is very easy to detect. More sophisticated attacks may bypass the detection of even the black-box RAG systems. Additionally, the results may be interpreted as if the problem lies in our implementation of the RAG pipeline. However, we want to highlight the fact that we followed the implementation suggested in the official Langchain documentation. It is likely that unexperienced developers follow that same documentation, resulting in a faulty RAG pipeline that crashes with such a simple attack.

Yes, Invisible injection can affect confidentiality, integrity, and availability of state-of-the-art RAG systems. Our demonstration highlights the need to carefully design these systems with cyber-secure principles.

7 Defenses

Although the presented injection strategies are diverse, many share syntactic or formatting patterns that can be detected using simple heuristics. Several attacks introduce structural or visual anomalies—imperceptible to humans but detectable via shallow pattern analysis or normalization. For example, diacritical mark stacking produces abnormal Unicode sequences that deviate from language norms. Statistical checks on combining marks or character frequencies can flag these. Homoglyph substitutions—though visually deceptive—leave traces in the form of anomalous Unicode blocks (e.g., Cyrillic in Latin text), which can be neutralized through character set filtering or canonical Unicode normalization (e.g., NFC).

Zero-width characters are rare in natural text and can be detected by scanning for a fixed set of Unicode code points. A blanket removal of such characters is often safe and effective. Bidirectional control characters (used in reordering attacks) introduce RTL/LTR switches that can be flagged by inspecting Unicode identifiers, especially when occurring mid-word or mid-token. Invisible injections using formatting tricks—e.g., transparent text, vanished styles, or zero-size fonts—can be detected via document parsing, targeting attributes like color, opacity, or tags (e.g., `<w:vanish>`). Similarly,

out-of-bound injections can be flagged through bounding-box analysis of rendered layout or structural metadata.

While these defenses are not foolproof, lightweight sanitization can mitigate many attacks at low cost. For more robust protection, advanced techniques like OCR-based pipelines [3] offer resilience against invisible perturbations but incur computational overhead and potential transcription errors. We also explored this OCR-based ingestion module (we tried two popular open-source OCR, i.e. EasyOCR and Tesseract), which extracts text from rendered document images, effectively bypassing many parser-level exploits. This approach blocks more 90% invisibility-based attacks, such as transparent or out-of-bound text, substantially reducing the attack success rate. However, OCR incurs significant computational overhead and may introduce transcription errors, limiting its practicality in high-throughput environments. Importantly, OCR is not a silver bullet: certain attacks, including diacritical manipulations and zero-size font injections, can still evade OCR-based defenses. Therefore, OCR should be considered a strong first layer in a multi-layered defense strategy rather than a standalone solution. These defensive insights and empirical results will be integrated into the revised manuscript to provide a balanced and actionable security perspective.

8 Related Works

Imperceptible Characters against NLP. Imperceptible perturbations to text have emerged as a powerful attack vector against NLP systems. Pajola et al. [18] introduced Zero-Width (ZeW) attacks, injecting invisible Unicode characters to bypass classifiers by major platforms like Google and Amazon. This was expanded [3] to include wide class of black-box attacks using encoding-level manipulations—zero-width, homoglyphs, and reordering—to break commercial NLP systems. Such attacks are also effective in subverting search engines and LLMs by stealthily altering queries and content, impacting retrieval, summarization, and ranking. [2] Our work took inspiration by these techniques, applying invisible injections to poison the document corpus of RAG systems.

Security of LLMs and RAG Systems. LLMs have shown impressive capabilities but also raise significant security concerns. Yao et al. [26] categorize the dual nature of LLMs as “The Good” (e.g., code analysis, privacy protection), “The Bad” (e.g., offensive misuse), and “The Ugly” (e.g., inherent vulnerabilities). LLMs have been

shown to be vulnerable to black-box jailbreak prompts that can bypass state-of-the-art safety measures [17]. RAG systems have also been studied, with attacks that can be used to create retrieval backdoors in RAG systems (TrojRAG [25]). Other attacks include prompt injections through documents [6] and knowledge corruption with specific question/answer pairs [28]. These works highlight the urgent need for robust defenses across both LLMs and their augmented systems.

9 Conclusion

Our analysis reveals significant vulnerabilities in RAG systems' data loading phase, stemming from widespread neglect of input sanitization. These weaknesses allow stealthy injection attacks that compromise the integrity of downstream language model outputs. Addressing these issues requires the disciplined application of established security practices to document ingestion pipelines. Furthermore, as AI systems increasingly rely on retrieved content for complex, multi-step reasoning, similar vulnerabilities may propagate and amplify across workflows. Future defenses must therefore advance beyond static sanitization to include context-aware filtering, provenance verification, and runtime anomaly detection to secure next-generation AI applications.

References

- [1] Christoph Auer, Maksym Lysak, Ahmed Nassar, Michele Dolfi, Nikolaos Livathinos, Panos Vagenas, Cesar Berrospi Ramis, Matteo Omenetti, Fabian Lindlbauer, Kasper Dinkla, Lokesh Mishra, Yusik Kim, Shubham Gupta, Rafael Teixeira de Lima, Valery Weber, Lucas Morin, Ingmar Meijer, Viktor Kuropiatnyk, and Peter W. J. Staar. 2024. Docling Technical Report. arXiv:2408.09869 [cs.CL] <https://arxiv.org/abs/2408.09869>
- [2] Nicholas Boucher, Luca Pajola, Ilia Shumailov, Ross Anderson, and Mauro Conti. 2023. Boosting big brother: Attacking search engines with encodings. In *Proceedings of the 26th International Symposium on Research in Attacks, Intrusions and Defenses*. 700–713.
- [3] Nicholas Boucher, Ilia Shumailov, Ross Anderson, and Nicolas Papernot. 2022. Bad characters: Imperceptible nlp attacks. In *2022 IEEE Symposium on Security and Privacy (SP)*. IEEE, 1987–2004.
- [4] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems* 33 (2020), 1877–1901.
- [5] Nicholas Carlini, Florian Tramer, Eric Wallace, Matthew Jagielski, Ariel Herbert-Voss, Katherine Lee, Adam Roberts, Tom Brown, Dawn Song, Ulfar Erlingsson, et al. 2021. Extracting training data from large language models. In *30th USENIX security symposium (USENIX Security 21)*. 2633–2650.
- [6] Harsh Chaudhari, Giorgio Severi, John Abascal, Matthew Jagielski, Christopher A Choquette-Choo, Milad Nasr, Cristina Nita-Rotaru, and Alina Oprea. 2024. Phantom: General trigger attacks on retrieval augmented language generation. *arXiv preprint arXiv:2405.20485* (2024).
- [7] Mauro Conti, Luca Pajola, and Pier Paolo Tricomi. 2023. Turning captchas against humanity: Captcha-based attacks in online social media. *Online Social Networks and Media* 36 (2023), 100252.
- [8] Gelei Deng, Yi Liu, Kailong Wang, Yuekang Li, Tianwei Zhang, and Yang Liu. 2024. Pandora: Jailbreak gpts by retrieval augmented generation poisoning. *arXiv preprint arXiv:2402.08416* (2024).
- [9] Tommi Gröndahl, Luca Pajola, Mika Juuti, Mauro Conti, and N Asokan. 2018. All you need is "love" evading hate speech detection. In *Proceedings of the 11th ACM workshop on artificial intelligence and security*. 2–12.
- [10] Yupeng Hou, Jiacheng Li, Zhankui He, An Yan, Xiushi Chen, and Julian McAuley. 2024. Bridging Language and Items for Retrieval and Recommendation. *arXiv preprint arXiv:2403.03952* (2024).
- [11] Jie Huang and Kevin Chen-Chuan Chang. 2022. Towards reasoning in large language models: A survey. *arXiv preprint arXiv:2212.10403* (2022).
- [12] Abhinav Kumar, Jaechul Roh, Ali Naseh, Marzena Karpinska, Mohit Iyyer, Amir Houmansadr, and Eugene Bagdasarian. 2025. OverThink: Slowdown Attacks on Reasoning LLMs. *arXiv e-prints* (2025), arXiv–2502.
- [13] David MJ Lazer, Matthew A Baum, Yochai Benkler, Adam J Berinsky, Kelly M Greenhill, Filippo Menczer, Miriam J Metzger, Brendan Nyhan, Gordon Pennycook, David Rothschild, et al. 2018. The science of fake news. *Science* 359, 6380 (2018), 1094–1096.
- [14] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems* 33 (2020), 9459–9474.
- [15] Nikolaos Livathinos, Christoph Auer, Maksym Lysak, Ahmed Nassar, Michele Dolfi, Panos Vagenas, Cesar Berrospi Ramis, Matteo Omenetti, Kasper Dinkla, Yusik Kim, Shubham Gupta, Rafael Teixeira de Lima, Valery Weber, Lucas Morin, Ingmar Meijer, Viktor Kuropiatnyk, and Peter W. J. Staar. 2025. Docling: An Efficient Open-Source Toolkit for AI-driven Document Conversion. arXiv:2501.17887 [cs.CL] <https://arxiv.org/abs/2501.17887>
- [16] Ian Markwood, Dakun Shen, Yao Liu, and Zhuo Lu. 2017. Mirage: Content masking attack against {Information-Based} online services. In *26th USENIX Security Symposium (USENIX Security 17)*. 833–847.
- [17] Anay Mehrotra, Manolis Zampetakis, Paul Kassianik, Blaine Nelson, Hyrum Anderson, Yaron Singer, and Amin Karbasi. 2024. Tree of attacks: Jailbreaking black-box llms automatically. *Advances in Neural Information Processing Systems* 37 (2024), 61065–61105.
- [18] Luca Pajola and Mauro Conti. 2021. Fall of Giants: How popular text-based MLaaS fall against a simple evasion attack. In *2021 IEEE European Symposium on Security and Privacy (EuroS&P)*. IEEE, 198–211.
- [19] Fábio Perez and Ian Ribeiro. 2022. Ignore previous prompt: Attack techniques for language models. *arXiv preprint arXiv:2211.09527* (2022).
- [20] Stephen Roller, Emily Dinan, Naman Goyal, Da Ju, Mary Williamson, Yinhan Liu, Jing Xu, Myle Ott, Kurt Shuster, Eric M Smith, et al. 2021. Recipes for building an open-domain chatbot. *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume* (2021).
- [21] Alexander Wei, Nika Haghtalab, and Jacob Steinhardt. 2023. Jailbroken: How does llm safety training fail? *Advances in Neural Information Processing Systems* 36 (2023), 80079–80110.
- [22] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems* 35 (2022), 24824–24837.
- [23] Laura Weidinger, Jonathan Uesato, Maribeth Rauh, Conor Griffin, Po-Sen Huang, John Mellor, Amelia Glaese, Myra Cheng, Borja Balle, Atoosa Kasirzadeh, et al. 2022. Taxonomy of risks posed by language models. In *Proceedings of the 2022 ACM conference on fairness, accountability, and transparency*. 214–229.
- [24] Qixue Xiao, Kang Li, Deyue Zhang, and Weilin Xu. 2018. Security risks in deep learning implementations. In *2018 IEEE Security and privacy workshops (SPW)*. IEEE, 123–128.
- [25] Jiaqi Xue, Mengxin Zheng, Yebowen Hu, Fei Liu, Xun Chen, and Qian Lou. 2024. Badrag: Identifying vulnerabilities in retrieval augmented generation of large language models. *arXiv preprint arXiv:2406.00083* (2024).
- [26] Yifan Yao, Jinhao Duan, Kaidi Xu, Yuanfang Cai, Zhibo Sun, and Yue Zhang. 2024. A survey on large language model (llm) security and privacy: The good, the bad, and the ugly. *High-Confidence Computing* (2024), 100211.
- [27] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, et al. 2023. Judging llm-as-a-judge with mt-bench and chatbot arena. *Advances in Neural Information Processing Systems* 36 (2023), 46595–46623.
- [28] Wei Zou, Runpeng Geng, Binghui Wang, and Jinyuan Jia. 2024. Poisonedrag: Knowledge corruption attacks to retrieval-augmented generation of large language models. *arXiv preprint arXiv:2402.07867* (2024).

A Appendix

The full appendix can be found in the public repository.

Ethical Statement

All experiments were conducted in accordance with ethical research guidelines. No proprietary or production systems were harmed or targeted during this study. The vulnerabilities identified in third-party open-source and commercial tools and libraries were responsibly disclosed to the respective parties prior to publication, following coordinated disclosure practices. Our goal is to raise awareness of these attack surfaces and promote the development of more secure and resilient RAG pipelines.

Acknowledgment

This work was produced as part of the "Governing Technology to Manage the Transition" (GoTMat) project. This project is funded by the European Union's Next Generation EU program under the National Recovery and Resilience Plan (PNRR) - Mission 4 "Education and Research" - Component 2 "From Research to Enterprise" - Investment 1.3. It is in accordance with Notice No. 341 of March 15, 2022, from the Ministry of University and Research, with reference to Project Code PE_00000014, "Security and rights in the Cyberspace (SERICS)", Spoke 9, CUP: B53C22003990006.

A.1 LLM-as-a-judge

For the experiment in Section 6.3, due to the large number of tests, we used an LLM-as-a-judge approach, where one LLM evaluates another's output. This enables automated annotation of vast data and is increasingly popular [27].

We used GPT-4o-mini as the judge model to evaluate answers from various RAG systems, determining if they contained specific characteristics indicative of successful attacks. The full definition of the prompts for all attacks can be found in the paper repository <https://github.com/pajola/PhantomText-Paper> (due to responsible disclosure, right now it is a google drive folder, and it will be replaced with a public repository upon paper publication). To evaluate this approach, we randomly sampled 100 RAG outputs for manual analysis and compared them to the LLM judge's evaluation. Only 3 samples were mislabeled. The 99% confidence Wilson Score Interval for accuracy is [0.8891, 0.9923].

To evaluate the performance of this approach, we randomly sampled 100 RAG outputs, which we manually analyzed. We compared our evaluation to the evaluation given by the LLM judge, and out of 100 samples only 3 samples were mislabeled. We computed the Wilson Score Interval for the accuracy of the model, which gives an interval of [0.8891, 0.9923] with confidence of 99%.

A.2 List of Software

Document Loaders:

- IBM's Docling toolkit: <https://github.com/docling-project/docling>
- Haystack by Deepset: <https://haystack.deepset.ai/>
- LangChain: <https://www.langchain.com/>
- LlamaIndex by LlamaIndex Inc.: <https://docs.llamaindex.ai/en/stable/>
- LLMSharpa by NLMatics: <https://github.com/nlmatics/llmsarpa>
- NotebookLM: <https://notebooklm.google/>.

A.3 RAG Implementation Details

White-box RAG Pipelines: These pipelines provided full access to internal components, including the retriever and language model. We tested three white-box configurations: **Llama 3.2** (3.21B parameters, Q4_K_M quantization—a 4-bit integer quantization with mixed-precision weight compression) with a local retriever; **Gemma 3** (27.2B parameters, Q4_0 quantization—standard 4-bit) with a local retriever (both at temperature 1.00); and **DeepSeek R1** accessed via API, also using a local retriever. For our local (white-box) RAG pipeline implementations, we followed the guidelines in

the official LangChain documentation and used a Chroma retriever with a default top $k = 4$ setting. In the public repository, we report the full system prompt used with the RAG pipelines that supported user-defined prompts.

Black-box RAG Pipelines: These pipelines involved using existing platforms or APIs where the internal workings of the retriever and language model were not directly accessible. We tested three black-box configurations: OpenAI GPT-4o (gpt-4o-2024-08-06), utilizing the assistant functionality on the OpenAI platform with default parameters (including a temperature of 1.00 and a default retrieval of 20); OpenAI o3-mini (o3-mini-2025-01-31), utilizing its built-in assistant functionality on the OpenAI platform with default parameters (including a temperature of 1.00 and a default retrieval of 20); and NotebookLM, a fully deployed RAG system with an online interface, based on Google's Gemini 2.0 model.

A.4 Experiment 3: Scenarios

A.4.1 Attack Scenarios. We evaluate nine attacks targeting RAG pipelines using a dataset of political biographies. Each scenario tests vulnerabilities via prompt injection, obfuscation, or document-level manipulation.

a1-Pipeline Failure. A 169 MB PDF embeds 97M characters out-of-bounds, causing infinite parsing or system crashes. *Obfuscation:* / *Injection:* out-of-bound text

a2-Reasoning Overload. A Sudoku puzzle is injected with font poisoning; success = excessive reasoning tokens. *Obfuscation:* font poisoning *Injection:* zero-size font, font poisoning

a3-Unreadable Output. Font poisoning and transparent text cause base64 gibberish replies about Draghi. *Obfuscation:* font poisoning *Injection:* transparent text, font poisoning

a4-Empty Statement Response. Obfuscation hides von der Leyen's birth info, leading to "unavailable" replies. *Obfuscation:* reordering characters, zero-width characters *Injection:* out-of-bound text

a5-Vague Output. Von der Leyen's bio altered with vague language via font poisoning. *Obfuscation:* font poisoning, transparent text *Injection:* font poisoning

a6-Bias Injection. Hidden praise in Draghi's bio biases responses. *Obfuscation:* camouflage, font poisoning, metadata, zero-size font *Injection:* font poisoning

a7-Factual Distortion. Fake event (cooking show) replaces real info in Evo Morales' bio. *Obfuscation:* font poisoning *Injection:* font poisoning

a8-Outdated Knowledge. Sassoli's death removed using homoglyphs, making the model say he is alive. *Obfuscation:* homoglyph characters, zero-width characters *Injection:* out-of-bound text

a9-Sensitive Data Disclosure. Prompt injection leaks another user's PII when querying for Elena Bianchi. *Obfuscation:* out-of-bound text *Injection:* /